



Meu Primeiro App com o GitHub Copilot ^{v2}

Guia de prompts passo a passo — app de freelancers com avaliação 360°
Node (backend) + React (frontend) · testes, cobertura e documentação
Projeto Social Garapuvu 2026 · Instrutor: Douglas Adriano Queiroz

Este guia foi feito para ser executado **ao vivo** na aula: você copia cada prompt no chat do GitHub Copilot (VS Code), na ordem apresentada, e mostra o sistema nascendo aos poucos. Cada prompt vem com **o que verificar e boas práticas**. Construímos backend e frontend, com a **pirâmide de testes** (unitário, integração e interface), **cobertura e documentação**.

✓ **Prompts validados.** Esta sequência foi executada de ponta a ponta antes da aula: o **backend gerado passa 13 testes** (7 unitários + 6 de integração) com **~95% de cobertura de linhas**. Ou seja, seguindo os prompts na ordem, você chega a um sistema que roda e é testável.

Setup usado nesta versão: projeto numa pasta `app/` com `backend/` e `frontend/` lado a lado; dados **em memória** (sem instalar banco); frontend **completo** com telas de Cadastro, Projetos, Avaliar e Perfil, em layout mobile-first.

1 Como usar este guia — as regras de ouro do Copilot

O Copilot é um copiloto: você pilota, ele acelera. A qualidade da resposta depende do seu pedido. Antes de começar, grave estas 6 regras:

- **Dê contexto.** Deixe abertos os arquivos relevantes e use `@workspace` no Copilot Chat para ele "enxergar" o projeto.
- **Peça em partes pequenas.** Um endpoint, um componente, um teste por vez — pedaços pequenos geram código mais confiável.
- **Diga a stack e as regras.** "Node + Express", "nota de 1 a 5", "e-mail único". Quanto mais específico, melhor.
- **Peça os testes junto.** Sempre que criar uma função, peça também os testes dela.
- **Leia antes de aceitar.** Você é responsável pelo código. Não entendeu? Pergunte "por que assim?".
- **Teste e versione a cada passo.** Rode os testes e faça `git commit` quando algo funcionar — assim é fácil voltar.

Dica de comandos do Copilot Chat: `/explain` explica um trecho, `/fix` sugere correção, `/tests` gera testes para a seleção, e `@workspace` considera todo o projeto no contexto.

2 O app que vamos construir: FreelaAvalia 360

Uma plataforma onde **contratantes** publicam projetos, **freelancers** se candidatam e, ao fim de um contrato, **os dois se avaliam** (nota de 1 a 5 + comentário). Essa avaliação mútua é a **avaliação 360°**: a reputação de cada pessoa é construída pelos dois lados.

Entidade	O que guarda	Regras principais
Usuário	nome, e-mail, papel (freelancer ou contratante)	e-mail único; papel válido
Projeto/ Contrato	título, descrição, contratante, freelancer, status	status: aberto → em andamento → concluído
Avaliação 360	de quem, para quem, nota (1–5), comentário	só após "concluído"; 1 avaliação por lado; nota entre 1 e 5

Stack: Backend em Node + Express; Frontend em React + Vite; testes com Vitest + Supertest + React Testing Library + Playwright; cobertura com o provider v8; qualidade com ESLint + Prettier.

3 Pré-requisitos e verificação das ferramentas

Antes dos prompts, confirme que tudo está instalado. Abra o terminal do VS Code e rode:

```
$ node -v      # deve mostrar v18, v20 ou superior
$ npm -v      # o gerenciador de pacotes do Node
$ git --version
$ code -v     # confirma o VS Code na linha de comando
```

Como saber se passou: cada comando deve imprimir um número de versão (ex.: v20.11.0). Se aparecer "command not found", a ferramenta não está instalada ou não está no PATH — instale antes de seguir.

No VS Code, confirme também a extensão **GitHub Copilot** instalada e o login feito com sua conta GitHub (ícone do Copilot na barra inferior, sem aviso de erro).

4 Bibliotecas que vamos usar (e por quê)

Tecnologia	Para que serve	Instalação
Express	Framework do backend: cria a API e os endpoints	<code>npm i express</code>
Vite + React	Base do frontend (telas)	<code>npm create vite@latest</code>
Vitest	Roda testes unitários e de integração	<code>npm i -D vitest</code>
Supertest	Testa a API (faz requisições nos endpoints)	<code>npm i -D supertest</code>
React Testing Library	Testa componentes de tela (interface)	<code>npm i -D @testing-library/react @testing-library/jest-dom</code>
Playwright	Testes E2E (fluxo real no navegador)	<code>npm i -D @playwright/test</code>
@vitest/coverage-v8	Mede a cobertura de testes	<code>npm i -D @vitest/coverage-v8</code>
ESLint + Prettier	Qualidade e formatação do código	<code>npm i -D eslint prettier</code>

Verificar bibliotecas instaladas: rode `npm ls --depth=0` para listar o que está no projeto, ou abra o arquivo `package.json` (seção `dependencies` e `devDependencies`). Cada `npm i` cria/atualiza esse arquivo e a pasta `node_modules/`.

5 A pirâmide de testes aplicada ao nosso app

Vamos testar em três níveis — muitos testes rápidos na base, poucos e lentos no topo:

Nível	No FreelaAvalia 360	Ferramenta
Unitário (base)	Lógica pura: validar nota (1–5), calcular a média das avaliações, checar e-mail	Vitest
Integração (meio)	A API funcionando: <code>POST /avaliacoes</code> salva? <code>GET /usuarios/:id</code> devolve a média?	Vitest + Supertest
Interface / E2E (topo)	Componente de nota renderiza; fluxo: criar contrato → concluir → avaliar	React Testing Library / Playwright

Leve daqui: escrevemos MUITOS testes de unidade (rápidos e baratos), ALGUNS de integração e POUCOS de ponta a ponta. Assim a suíte é rápida e confiável.

6 A sequência de prompts (execute nesta ordem)

A ordem importa: o Copilot rende muito melhor quando o projeto já tem estrutura e ele "vê" os arquivos anteriores. Faça um `git commit` ao fim de cada fase.

Fase 0 — Preparar o projeto

```
$ mkdir freelavalua && cd freelavalua
$ git init
$ npm init -y
$ code .           # abre o VS Code nesta pasta
```

PROMPT 0.1 — CONTEXTO PARA O COPILOT

@workspace Vamos criar um app chamado FreelaAvalia 360: uma plataforma de freelancers com avaliação 360° entre freelancer e contratante. Backend em Node + Express, frontend em React + Vite, testes com Vitest, Supertest, React Testing Library e Playwright. Proponha uma estrutura de pastas simples (backend/ e frontend/) e um README inicial explicando o projeto. Ainda não escreva código de funcionalidade – só a estrutura e o README.

Verificar: foram criadas as pastas backend/ e frontend/ e um README.md. Leia o README: ele descreve o app corretamente?

Fase 1 — Backend: modelo e regras (lógica pura = testes unitários)

```
$ cd backend && npm init -y
$ npm i express
$ npm i -D vitest supertest @vitest/coverage-v8
```

PROMPT 1.1 — REGRAS DE NEGÓCIO ISOLADAS

@workspace Crie o arquivo backend/src/regras.js com funções puras (sem banco, sem Express) e comentários JSDoc explicando cada uma:

- validarNota(nota): retorna true se for inteiro entre 1 e 5.
- emailValido(email): valida formato básico de e-mail.
- mediaAvaliacoes(lista): recebe uma lista de notas e retorna a média com 1 casa decimal (0 se vazia).
- podeAvaliar(contrato): só permite avaliar se o status do contrato for "concluido".

Explique cada função em uma frase.

PROMPT 1.2 — TESTES UNITÁRIOS DESSAS REGRAS

@workspace Gere testes unitários com Vitest para backend/src/regras.js, no arquivo backend/src/regras.test.js. Cubra casos válidos, inválidos e de borda: nota 0, 1, 5, 6 e valores não inteiros; e-mail com e sem @; média de lista vazia e de várias notas; podeAvaliar para contrato "aberto" e "concluido".

```
$ npx vitest run           # roda os testes uma vez
$ npx vitest run --coverage # roda + relatório de cobertura
```

Verificar: o terminal mostra os testes **passando** (verde). O relatório de cobertura mostra `regras.js` perto de 100%. Boa prática: **testar as bordas** (nota 0 e 6) é o que pega a maioria dos bugs.

Fase 2 — Backend: a API (endpoints = testes de integração)

PROMPT 2.1 — ARMAZENAMENTO SIMPLES E SEPARADO

@workspace Crie `backend/src/repositorio.js`: um armazenamento em memória (objetos/arrays) para `usuarios`, `contratos` e `avaliacoes`, com funções de criar e buscar. Mantenha-o separado da API (para facilitar os testes). Comente cada função com JSDoc.

PROMPT 2.2 — A APLICAÇÃO EXPRESS

@workspace Crie `backend/src/app.js` exportando uma app Express (sem dar `app.listen` aqui — isso facilita os testes). Use as regras de `regras.js` e o `repositorio.js`. Endpoints:

- `POST /usuarios` (cria usuário; e-mail único e válido; papel "freelancer" ou "contratante")
- `POST /contratos` (cria contrato entre contratante e freelancer)
- `PATCH /contratos/:id/concluir` (muda status para "concluído")
- `POST /avaliacoes` (só se o contrato estiver concluído; nota 1–5; 1 por lado)
- `GET /usuarios/:id` (retorna o usuário com a média de avaliações recebidas)

Valide as entradas e retorne os status HTTP corretos (201, 400, 404). Comente cada rota.

PROMPT 2.3 — SEPARAR O SERVIDOR

@workspace Crie `backend/src/server.js` que importa a app de `app.js` e chama `app.listen(3001)`. Adicione no `package.json` os scripts `"start": "node src/server.js"` e `"test": "vitest run"` e `"coverage": "vitest run --coverage"`.

PROMPT 2.4 — TESTES DE INTEGRAÇÃO DA API

@workspace Gere testes de integração com Vitest + Supertest em `backend/src/app.test.js`, importando a app de `app.js`. Teste o fluxo feliz e os erros: criar usuários, criar contrato, tentar avaliar antes de concluir (deve dar 400), concluir, avaliar dos dois lados, e conferir que `GET /usuarios/:id` retorna a média correta. Inclua um teste de e-mail duplicado (400).

```
$ npx vitest run --coverage
```

Verificar: todos os testes passam e a cobertura de `app.js` sobe. Teste também "à mão": rode `npm start` e, no Postman/Insomnia, faça um `POST /usuarios` e um `GET /usuarios/:id`. Boa prática: **separar app de server** (Prompt 2.2/2.3) deixa a API testável sem subir a porta.

Fase 3 — Frontend: React + Vite (começando pelo mobile-first)

Antes de codar a tela: o conceito mobile-first

Mobile-first significa **projetar primeiro para a tela pequena (celular) e depois crescer para telas maiores** (tablet, desktop) — e não o contrário. A maioria das pessoas acessa apps pelo celular, então começamos pelo essencial que cabe na tela estreita e, conforme sobra espaço, adicionamos colunas e detalhes.

Analogia da mala de viagem: quando você arruma uma mala pequena, é obrigado a levar só o essencial e bem organizado. Se depois te dão uma mala maior, é fácil espalhar as coisas com folga. O contrário é um pesadelo: pegar tudo de uma mala grande e tentar espremer numa pequena não cabe. Mobile-first é arrumar a mala pequena primeiro.

Na prática, o CSS mobile-first escreve o estilo base para o celular e usa `@media (min-width: ...)` para **adicionar** ajustes em telas maiores. Benefícios: foco no essencial, melhor desempenho no celular e menos retrabalho.

```
$ cd ../frontend
$ npm create vite@latest . -- --template react
$ npm i
$ npm i -D vitest @testing-library/react @testing-library/jest-dom jsdom @vitest/coverage-v8
```

PROMPT 3.0 — BASE DE ESTILO MOBILE-FIRST

@workspace Configure o frontend com abordagem MOBILE-FIRST. Em frontend/src/index.css: defina os estilos base pensando primeiro no celular (largura ~360px) – fonte legível, botões grandes o suficiente para o toque (mín. 44px de altura), espaçamentos confortáveis e conteúdo em uma coluna. Depois use `@media (min-width: 768px)` e `(min-width: 1024px)` apenas para ADICIONAR layout de mais colunas em telas maiores. Use a paleta do Projeto Garapuvu: verde #2E7D32 (primária), verde-escuro #14401A, amarelo #F9A825 (destaque), fundo claro #F1F8E9 e texto #1B3A1B. Comente o que cada breakpoint faz.

Verificar: abra o app e reduza a janela do navegador (ou use o F12 → modo dispositivo). No celular, o conteúdo fica em uma coluna, sem rolagem lateral e com botões fáceis de tocar; ao alargar, o layout ganha colunas. Boa prática: **comece testando no tamanho do celular** — é onde a maioria dos usuários está.

PROMPT 3.1 — COMPONENTE DA NOTA (ESTRELAS)

@workspace Crie frontend/src/components/Estrelas.jsx: um componente React que recebe a prop "valor" (1 a 5) e mostra estrelas preenchidas/vazias, e uma prop opcional onChange para selecionar a nota clicando. Comente as props. Não use bibliotecas externas.

PROMPT 3.2 — TELA DE AVALIAÇÃO CONSUMINDO A API

@workspace Crie frontend/src/pages/Avaliar.jsx: um formulário para enviar uma avaliação 360 (escolher nota com o componente Estrelas + comentário) que faz POST em `http://localhost:3001/avaliacoes`. Trate carregando, sucesso e erro (mensagens claras para o usuário). Use fetch. Comente o fluxo.

PROMPT 3.3 — PERFIL COM A MÉDIA RECEBIDA

@workspace Crie frontend/src/pages/Perfil.jsx que busca GET `/usuarios/:id` e mostra o nome, o papel e a média de avaliações usando o componente Estrelas. Trate o estado de carregando e de erro.

Verificar: rode `npm run dev (frontend)` e `npm start (backend, noutro terminal)`. A tela de avaliar envia e mostra sucesso? O perfil mostra a média? Boa prática: sempre tratar **carregando** e **erro**, não só o "caminho feliz".

Fase 4 — Testes de interface e E2E

PROMPT 4.1 — TESTE DE COMPONENTE (INTERFACE)

@workspace Gere um teste com Vitest + React Testing Library em frontend/src/components/Estrelas.test.jsx: renderiza com valor 3 e verifica que há 3 estrelas preenchidas; simula um clique na 5ª estrela e verifica que o onChange foi chamado com 5. Configure o Vitest para usar o ambiente jsdom.

```
$ npx vitest run --coverage # testes de interface + cobertura do front
$ npm i -D @playwright/test && npx playwright install
```

PROMPT 4.2 — TESTE E2E (FLUXO COMPLETO)

@workspace Crie um teste E2E com Playwright em frontend/e2e/avaliacao.spec.js que: abre a tela de avaliar, seleciona a nota 5, escreve um comentário, envia e verifica que aparece a mensagem de sucesso. Crie também um playwright.config.js apontando para http://localhost:5173.

Verificar: npx vitest run (componentes) e npx playwright test (E2E) passam. Boa prática: use **seletores estáveis** (ex.: data-testid) para o robô não quebrar quando o visual mudar.

Fase 5 — Cobertura, relatório e CI

PROMPT 5.1 — RELATÓRIO DE COBERTURA LEGÍVEL

@workspace Configure o Vitest (vitest.config.js) para gerar cobertura com o provider "v8" nos formatos "text" (no terminal) e "html" (pasta coverage/). Defina limites mínimos (thresholds): 80% de linhas e funções. Explique, em comentário, o que cada limite significa.

```
$ npx vitest run --coverage
$ open coverage/index.html # abre o relatório visual (no Windows: start)
```

PROMPT 5.2 — RODAR TUDO NO GITHUB ACTIONS

@workspace Crie .github/workflows/ci.yml que, a cada push, instala as dependências e roda os testes com cobertura do backend e do frontend. Se a cobertura ficar abaixo do limite, o build deve falhar.

Verificar: o relatório HTML mostra, por arquivo, a % de linhas cobertas e destaca em vermelho as linhas **não testadas**. Boa prática: cobertura **mostra o que faltou testar** — não prova ausência de bugs (100% verde com asserção fraca não vale nada).

Fase 6 — Documentação

PROMPT 6.1 — DOCUMENTAR MÉTODOS E API

@workspace Revise regras.js, repositorio.js e app.js e garanta que TODA função e TODA rota têm comentário JSDoc (o que faz, parâmetros, retorno). Em seguida, atualize o README com: o que é o app, tecnologias usadas, como instalar, como rodar (backend e frontend), como executar os testes e como ver a cobertura, e uma tabela dos endpoints da API.

Verificar: o README permite que outra pessoa rode o projeto do zero seguindo só ele. Boa prática: documentação errada engana — revise se ela bate com o que o código faz.

7 Boas práticas ao construir com IA (resumo)

- **Contexto primeiro:** comece cada fase com um prompt que lembra o Copilot do objetivo e da stack (`@workspace`).
- **Incremental:** regras → API → testes → frontend → E2E → cobertura → docs. Nunca peça "o app inteiro" de uma vez.
- **Teste sempre o que a IA gera:** código sem teste é rascunho. Rode a suíte a cada passo.
- **Nunca aceite o que não entende:** use `/explain` . Você assina o código, não a IA.
- **Segurança:** nada de senhas/segredos no código. Use variáveis de ambiente (`.env`) e um `.gitignore` (ignore `node_modules/` e `.env`).
- **Versione:** `git add . && git commit -m "..."` ao fim de cada fase que funcionar.

8 Checklist da aula e conceitos-chave

Conceito	O que é	Onde aparece no app
Teste unitário	Testa uma função isolada	regras.js (nota, média, e-mail)
Teste de integração	Peças funcionando juntas (API + regras + repo)	app.test.js (endpoints)
Teste de interface/ E2E	Componente e fluxo do usuário	Estrelas.test.jsx, avaliacao.spec.js
Cobertura	% do código exercitado pelos testes	relatório HTML em coverage/
Threshold	Limite mínimo de cobertura que barra o build	vitest.config.js + CI
Mobile-first	Projetar primeiro para o celular e crescer para telas maiores	index.css (Prompt 3.0) e todas as telas
Documentação	JSDoc nos métodos + README	todo o projeto

Para apresentar aos alunos: mostre a pirâmide viva — rode primeiro os unitários (instantâneos), depois a integração e por fim o E2E (mais lento, abre o navegador). Termine abrindo o relatório de cobertura e apontando uma linha vermelha (não testada): é o mapa do que ainda falta.

9 Referências

GitHub Copilot Docs	docs.github.com/copilot
Node.js / Express	nodejs.org · expressjs.com
Vite / React	vite.dev · react.dev
Vitest (testes e cobertura)	vitest.dev
Supertest / Testing Library / Playwright	github.com/ladjs/supertest · testing-library.com · playwright.dev